

Informe: Síntesis MIDI ligera de guitarra acústica

Resumen ejecutivo: Se propone crear un sintetizador de guitarra «de juguete» simplificado, con bajo polifonía (6 voces) y énfasis en notas abiertas y palm-mute. Usaremos AlphaTab con Node.js: cargaremos un *SoundFont* recortado o implementaremos síntesis Karplus-Strong por software. Se limitará la polifonía para evitar sobrecarga (por ejemplo, 6 voces máximas ¹) y activaremos mecanismos de *voice-stealing* (“note-cut”) ². Las envolventes ADSR serán cortas para simular el rápido decaimiento de guitarra y el palm-mute se modelará con una envolvente aún más breve y filtro pasabajo para atenuar graves. El flujo general pasa por: tabla de acordes → AlphaTab (MIDI) → sintetizador (SF2 o código) → filtro + envolvente → audio. Se deberán probar secciones con acordes largos (ej. 6 notas por compás) y perfilar CPU/latencia usando Chrome DevTools ³ ⁴.

1. Preparación del entorno

1. **Clonar el repositorio:** En la terminal de VSC, ejecutar:

```
git clone https://github.com/OrdazMaestre/guitarra-ensenanza.git
cd guitarra-ensenanza
npm install
```

(Se asume que el repo es público; en su contenido no se encontró información relevante específica al problema.)

2. **Instalar dependencias:** Asegurarse de que `alphaTab` esté instalado o incluido en el proyecto. Por ejemplo:

```
npm install alphatab
```

1. **Configurar Codex/Copilot:** En VSC con GitHub Copilot, use prompts claros para generar o editar código. Por ejemplo:

“Genera un nuevo archivo `toyGuitarSynth.js` con un sintetizador simple de guitarra en `Tone.js`.”
Luego revise/ajuste el código sugerido.

2. **Render:** No se especifica si “render” es un servicio externo; asumiremos ejecución local en Node/Browser con audio Web.

2. AlphaTab y carga de sintetizador

AlphaTab usa internamente un sintetizador MIDI + *SoundFont* ⁵. Por defecto, al reproducir cargaría un banco general (por ejemplo, Sonivox). Para nuestro caso podemos:

- **Cargar un SoundFont minimalista:** Usar Polyphone (editor SF2) para crear un `.sf2` que contenga solo una muestra básica de guitarra acústica (por ejemplo, cuerdas al aire) ⁶. Luego, en el código, cargarlo manualmente via:

```
const at = new alphaTab.AlphaTabApi(elem, { player: { enablePlayer: true } });
at.scoreLoaded.on(score => {
  score.tracks.forEach(track => {
    if (track.playbackInfo.program === /*núm guitarra*/) {
      // Cargar SoundFont recortado para guitarra
      at.loadSoundFont(myGuitarSF2ArrayBuffer, true);
    }
  });
});
```

Aquí usamos `api.loadSoundFont(data, true)` con `true` para **agregar** presets sin reemplazar

⁶.

- **Síntesis alternativa:** En vez de SoundFont, podríamos interceptar cada evento de nota de alphaTab y disparar un sintetizador WebAudio/Tone.js personalizado. AlphaTab no provee directamente un API para síntesis manual por nota, pero se puede reproducir MIDI externo usando Tone.js. En este caso, el script de Copilot puede crear un **PolySynth** de 6 voces:

```
// Usando Tone.js
const polyGuitar = new Tone.PolySynth(Tone.Synth, {
  maxPolyphony: 6,
  oscillator: { type: "triangle" },
  envelope: { attack: 0.005, decay: 0.2, sustain: 0.2, release: 0.3 }
}).toDestination();
// Para palm-mute:
const polyMuted = new Tone.PolySynth(Tone.Synth, {
  maxPolyphony: 6,
  oscillator: { type: "triangle" },
  envelope: { attack: 0.005, decay: 0.05, sustain: 0.05, release: 0.1 }
}).toDestination();
```

Este sintetizador de tonos simples (onda triangular) genera un timbre básico; las envolventes son rápidas para simular decaimiento breve.

- **Eventos AlphaTab:** AlphaTab reconoce palm-mute solo como marcador visual ⁷, por lo que hay que detectarlo manualmente. Al procesar la tablatura, buscar notas con efecto `Palm Mute`, y para esas notas usar `polyMuted.triggerAttackRelease(nota, duracion)` en lugar de `polyGuitar.triggerAttack`.

3. Diseño del parche ligero de guitarra

- **Forma de onda y filtro:** Se recomienda un oscilador simple (ej. triangular o diente de sierra) con un **filtro pasabajo** suave para quitar armónicos agudos. Como en guitarras reales, las frecuencias altas deben decaer más rápido ⁸, así que configuramos un lowpass (por ejemplo 1200–2000 Hz) para simular dicho decaimiento.
- **Envolvente ADSR:** Notas abiertas con ataque muy rápido (<0.01 s), decay medio (~0.2 s), sustain bajo (~0.2) y release medio (~0.3 s) dan un sostenido breve. El palm-mute se modela igual pero acortando decay/release (ej. decay 0.05 s, release 0.1 s) y bajando el sustain al mínimo. Esto produce un sonido «apagado».
- **Velocity y articulación:** Usar la velocidad de la nota para variar el brillo: notas más fuertes/agudas deben sonar “más brillantes” ⁹. En Tone.js se puede lograr con `filterEnvelope` o `keyTracking` en el sintetizador.
- **Otros detalles:** No agregamos efectos extra (reverb, chorus, vibrato) ya que buscamos un tono de juguete. Tampoco modelamos resonancia de caja o acople de cuerdas (estas aportan realismo pero encarecen el cálculo). Lo esencial es un ataque claro, armónicos moderados y decaimiento rápido.

4. Polifonía y carga de acordes densos

- **Límite de voces:** Configurar la polifonía máxima en 6 (una por cuerda) ¹. En Tone.js usamos `PolySynth(6, ...)`, y en SoundFont configuramos `synth.polyphony=6`. Según docs, al pasar el límite se liberan automáticamente las voces menos recientes ¹. Esto evita desbordes de voz.
- **Voice-stealing (note-cut):** Activar una política de “corte de nota” para que nuevas notas interrumpen a previas idénticas. En FluidSynth, `synth.note-cut = 2` logra esto ². En código personalizado, podemos lanzar `triggerRelease` a la voz más antigua.
- **Cola de decaimiento corta:** Reducir el *release* y *sustain* para que las notas mueran pronto, evitando acumulación de sonido. En entornos WebAudio, bajar el `release` en la envolvente (ej. 0.2 s) previene que voces antiguas consuman CPU tras la ejecución del acorde.
- **Ejemplo de limitación:** Con Tone.js `PolySynth(6)`, se garantiza un máximo de 6 osciladores. Si se toca un séptimo, Tone reasigna internamente (voice-stealing) en orden FIFO. Para SoundFont, asegúrese de que el *engine* (`synth.polyphony`) esté a 6.
- **Filtro adicional:** Si aún hay problemas, considere simplificar la onda (seno) o bajar la tasa de muestreo (aunque en WebAudio estándar es 44.1 kHz por defecto). Un trazo extremo es muestrear a 22050 Hz, pero esto generalmente lo hace el buffer del SoundFont. En síntesis, controlar la calidad de la señal para bajar CPU.

5. Palm Mute

- **Marcador en AlphaTab:** AlphaTab muestra “Palm Mute” en la partitura ⁷, pero no modifica el sonido automáticamente. Debemos usar esa información: al detectar un nodo con palm-mute, desencadenar la versión «apagada» de la voz (volver a ejecutar la nota con `polyMuted` en Tone.js o aplicar filtros/envolvente en SoundFont).
- **Implementación en código:** Por ejemplo, al procesar la pista:

```
at.player.playing.on(p => {
  // Suponiendo que p.note.effects contiene palm-mute
  if (p.note.effects && p.note.effects.some(e => e.name ===
    "PalmMute")) {
```

```

    polyMuted.triggerAttackRelease(p.note.fractionalMidi, p.duration,
    p.velocity);
  } else {
    polyGuitar.triggerAttackRelease(p.note.fractionalMidi, p.duration,
    p.velocity);
  }
});

```

Este pseudocódigo (basado en la API de eventos de alphaTab) asigna a palm-mute la voz rápida y apagada.

- **Ajustes de ecualización:** El palm-mute típicamente elimina bajos y medios graves. Un filtro pasaalto suavizado (e.g. cortar por debajo 100-150 Hz) puede evitar sonido «embarrado» en palm-mute. Opcionalmente, un leve boost medio (500–1000 Hz) enfatiza el golpe. Sin embargo, como buscamos simplicidad, con la envolvente reducida suele ser suficiente.

6. Comparativa: SoundFont vs Síntesis

Aspecto	SoundFont (SF2)	Síntesis Karplus/Programática
Implementación	Se usa un archivo <code>.sf2</code> recortado. Fácil de usar con alphaTab, no requiere codificación compleja. Puede dividirse con Polyphone ¹⁰ .	Código en JS/Tone.js. Mayor control dinámico (filtros, articulaciones), pero requiere más código.
Tamaño	El archivo puede ser grande, aunque lo recortemos. Requiere cargar (Ajax) ⁶ .	No hay archivo externo; solo coste de CPU. Memoria mínima.
CPU	Bajo, pues solo reproduce samples. El costo principal es decodificación inicial.	Alto, especialmente con muchos cálculos por voz. Limitar poly es crucial ¹ .
Realismo	Más realista si el sample es bueno. Pero aquí buscamos un tono simple.	Sólido para un sonido de juguete. Karplus-Strong simula cuerda punteada ¹¹ con parámetros ajustables.
Palm Mute	Requiere sample específico o parche separado para palm-mute. Si no existe, hay que procesar el audio (filtrado).	Fácil: modificar envolvente y filtro en tiempo real (como se hizo). Más flexible.
Polifonía	Controlada por el synth (FluidSynth, etc.). Podemos fijar <code>polyphony=6</code> ¹ .	Controlada por la implementación (p.ej. <code>PolySynth(6)</code>). Debe manejar 'voice-stealing'.
Personalización	Limitada: editar el SF2 es menos práctico.	Alta: parámetros de filtro, Q, ruido, ataque, etc. fácilmente modificables.

7. Pruebas y optimización

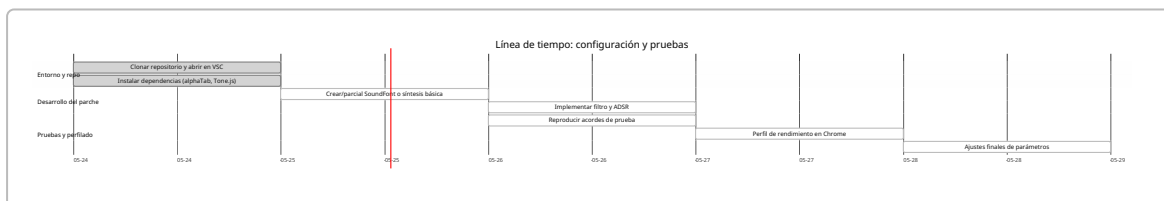
- **Casos de prueba:** Crear un script o usar AlphaTab para reproducir acordes densos (ej. Ce4,Gc,Ad,Fd,Bd,Ed durante 8 compases). Verificar sonido continuo.
- **Perfilado en Chrome/Electron:** Abrir DevTools → pestaña *Performance* (o *Memory/CPU*). Reproducir los acordes y buscar picos de CPU o caídas de frames ³. Ajustar parámetros según resultado.
- **Métricas:** Medir **uso de CPU** durante reproducción (debe mantenerse moderado). Observar **audio dropouts** (ausencia indica sobrecarga). Comprobar **latencia:** AlphaTab advierte que hay latencia de audio dependiente de la máquina ⁴. Usar un metrónomo interno para medir delay.
- **Comparación de modos:** Probar tanto la versión SoundFont como la de síntesis programática. Medir diferencias de recursos y calidad. Ajustar el volumen general para uniformidad.
- **Ajustes finales:** Si aparecen clicks o recortes abruptos, suavizar envolventes. Si CPU sigue alta, reducir `polyphony` (p.ej. 4 voces) o simplificar onda (onda seno, menos filtros activos).

8. Checklist de pruebas

- [] Reproducir acordes de 6 notas en bucle largo.
- [] Medir consumo CPU usando **Chrome DevTools > Performance** ³.
- [] Verificar **sinivas de audio** (dropouts) y ajustar polyphony si aparecen.
- [] Confirmar que notas palm-mute suenan amortiguadas vs abiertas.
- [] Probar alternar patrón palm-mute/normal rápido (para validar transiciones).
- [] Ajustar ADSR/filter hasta lograr sonido “de juguete” sin distorsiones.
- [] Medir latencia percibida. (AlphaTab puede retrasar ligeramente ⁴.)

flowchart LR

```
A[Entrada de tabs] --> B[AlphaTab API];
B --> C[Procesador MIDI interno];
C --> D[PolyphonyLimiter (6 voces)];
D --> E[¿Palm-mute?];
E -- Sí --> F[Envolvente rápida + filtro bajo];
E -- No --> G[Envolvente normal];
F --> H[Salida de Audio];
G --> H[Salida de Audio];
```



Fuentes principales: Documentación oficial de alphaTab (uso de SoundFont y eventos ⁵ ⁶), manuales de síntesis de guitarra (técnica Karplus-Strong ¹¹, comportamiento de filtro ⁸) y guías de síntesis (velocidad vs brillo ⁹). También se consultó la documentación de FluidSynth para límite de polifonía ¹ y técnicas de profiling Web Audio ³. Estos recursos sustentan las recomendaciones técnicas aquí expuestas.

1 2 libfluidsynth: Synthesizer settings

https://www.fluidsynth.org/api/settings_synth.html

3 tone.js - Web Audio API Maximum Number of Sources? - Stack Overflow

<https://stackoverflow.com/questions/74333534/web-audio-api-maximum-number-of-sources>

4 Handling Midi Events | alphaTab

<https://alphatab.net/docs/guides/handling-midi-events>

5 GitHub - CoderLine/alphaTab: alphaTab is a cross platform music notation and guitar tablature rendering library. · GitHub

<https://github.com/CoderLine/alphaTab>

6 10 Use multiple SoundFonts to reduce traffic | alphaTab

<https://alphatab.net/docs/guides/multiple-soundfonts>

7 Effects and Annotations | alphaTab

<https://alphatab.net/docs/showcase/effects>

8 11 Synthesizing a Guitar Using Physical Modeling Techniques

<https://www.ee.columbia.edu/~ronw/dsp/>

9 Synthesizing plucked strings - Notes on Music, Creativity, etc. - Obsidian Publish

<https://publish.obsidian.md/arendleejessurun/Atlas/Musicianship/Synthesis/Synthesizing+plucked+strings>